

Characterizing Readability Issue Patterns and the Role of Prompt Design in LLM-Generated Code

Hengzhi Ye, Fengyuan Ran, Weiwei Xu, Minghui Zhou

Abstract—Large Language Models (LLMs) are increasingly changing how code is produced, but generated code still requires human review and validation before it can be adapted or integrated into real-world projects. This makes the readability of LLM-generated code a critical concern. Existing studies have mainly focused on functional correctness and task completion of LLM generated code, leaving open questions about whether it is readable, how its readability fails, and to what extent prompt design can improve it. We therefore investigate the readability of LLM-generated code by examining how it compares with human-written code, what distinct issue patterns it exhibits, and how prompt design is associated with these readability outcomes. We first construct a readability assessment model that integrates textual, structural, program, and visual features. Using this model, we compare human-written code with code generated by representative frontier LLMs across 2,735 scenarios derived from World of Code (WoC) and LeetCode. We further characterize readability issue patterns using thematic analysis and examine prompt design associations through controlled prompt-variant experiments. Our results show that current LLMs produce code that is comparable to human-written code in overall readability. However, this aggregate similarity masks distinct issue patterns, including excessive complexity, redundant comments, and unknown API usage. Prompt analysis further shows that *function signature*, *constraints*, and *style description* are most strongly associated with code readability, although the overall role of prompt design remains bounded. These findings reveal latent readability debt in AI-assisted programming, identify prompt design as a lightweight starting point for improving generated-code readability, and motivate automated support for detecting and mitigating readability issues in future development workflows.

Index Terms—Code readability, code generation, prompt engineering, LLM

I. INTRODUCTION

Code readability has long been critical to software engineering, as developers have to first understand code before they can review, modify, reuse, or extend it [1]. Readable code reduces comprehension effort, lowers maintenance burden, and helps prevent technical debt [2], [3].

The widespread use of Large Language Models (LLMs) has changed how code is produced. However, LLM-generated code still needs human oversight, as it may contain hallucinations, outdated knowledge, or subtle vulnerabilities [4], [5]. Developers are therefore required to judge whether generated code can be trusted to be adapted or integrated into development

workflows. When such code is hard to read, the cost of review may offset the productivity gains of AI assistance [6].

Existing research has primarily evaluated LLM-generated code through functional correctness and task completion [7], [8]. Although essential, these criteria say little about how readable generated code really is once it enters human review and maintenance workflows. A large-scale comparison with human-written code is therefore important to establish a practical baseline. However, overall readability scores alone cannot reveal how readability fails, as LLM-generated code may be difficult to read for reasons different from human-written code. Furthermore, although prompt design is the primary way users steer LLM outputs, little is known about which prompt dimensions are associated with more readable generated code. Understanding these questions is important for improving the reliability and maintainability of AI-assisted software development.

Thus, this paper investigates the readability of LLM-generated code through three research questions:

- **RQ1:** How does the overall readability of LLM-generated code compare with human-written code?
- **RQ2:** What readability issue patterns distinguish LLM-generated code from human-written code?
- **RQ3:** What role does prompt design play in shaping the readability of LLM-generated code?

We started by constructing a readability assessment model that integrates textual, structural, program, and visual features into a unified framework. Using this model, we compare human-written code with code generated by representative frontier LLMs available at the time of our study across 2,735 scenarios extracted from World of Code (WoC) [9] and LeetCode [10]. We then examined the readability issues LLM-generated code exhibits and studied whether their patterns differ from those in human-written code through a hybrid deductive–inductive thematic analysis. Finally, we analyzed the relationship between prompt dimensions and generated-code readability across 5,248 samples produced from controlled prompt variants.

Our results show that LLM-generated code is broadly comparable to human-written code in overall readability. However, a critical divergence exists behind superficial similarity: LLM-generated code exhibits distinct readability issue patterns, including unnecessary complex structures, redundant comments, unknown API usage, etc. Further prompt analysis shows that *function signature*, *constraints*, and *style description* are most associated with the readability of generated code, while the overall role of prompt design is bounded. These findings reveal how LLM-generated code differs from human-written code

Hengzhi Ye, Weiwei Xu and Minghui Zhou are with the School of Computer Science, Peking University, Beijing 100871, China (e-mail: hzye@stu.pku.edu.cn, xuww@stu.pku.edu.cn, zhmh@pku.edu.cn). Fengyuan Ran is with Wuhan University, Wuhan 430072, China (e-mail: rfy_reflow@whu.edu.cn).

in readability, expose latent readability debt in AI-assisted programming, and identify prompt design as a lightweight starting point for readability improvement, offering insights for managing and improving LLM-generated code.

In summary, this paper makes the following contributions:

- An **empirical study** of LLM-generated code readability, showing that LLM-generated code is broadly comparable to human-written code in aggregate readability but differs in concrete readability issue patterns;
- An empirical **analysis of prompt design**, identifying the prompt dimensions associated with more readable code while showing that their role remains bounded;
- A **scalable benchmark** for comparing LLM-generated and human-written code, consisting of 2,735 task prompts and corresponding human implementations from real-world software practices and programming competitions;
- A comprehensive **readability assessment model** that integrates textual, structural, program, and visual features to quantify code readability at scale.

In Section II, we review related work. We illustrate the overall methodology in Section III and explain our readability assessment model in Section IV. Sections V to VII detail the results for each research question. We discuss implications and limitations in Section VIII and conclude in Section IX.

II. RELATED WORK

A. LLM Code Generation

The application of LLMs to code generation substantially enhanced automated programming capabilities. Not only have general LLMs like ChatGPT been widely deployed for code generation [11], but fine-tuned models such as Codex [8] and StarCoder [12] have also emerged to achieve competitive performance across various programming languages. Integrated code agents like Copilot [13] and Cursor [14] further enhance developer productivity. Surveys indicate that Copilot is behind an average of 46% of developers' code across all programming languages [15], highlighting the profound impact of LLM-generated code on software development.

Researchers have focused on examining whether LLM-generated code is functionally correct, secure, and aligned with developer expectations. For example, HumanEval [8] utilizes reference solutions and unit tests to quantify the functional correctness of LLM-generated code. Additional evaluation criteria have also examined LLM-generated code in terms of vulnerability risks, privacy protection, and copyright compliance [16], [17]. However, little attention is devoted to the readability of the generated code despite its importance to long-term software lifecycle.

B. Code Readability

Since the emergence of programming, code readability has become an implicit code quality criterion.

Buse and Weimer [18] developed the first readability model by mapping 25 structural features, such as indentation and line length, to human-annotated code. Their findings established

a positive correlation between readability scores and overall code quality. Subsequent studies refined structural metrics, with Posnett et al. [19] achieving higher accuracy through a simpler three-feature model and Johnson et al. [20] exploring how nesting and looping constructs influenced readability. A significant advancement occurred as Scalabrino et al. [21], [22] integrated textual and semantic features with structural ones to capture the dual nature of code as both logic and language.

More recently, researchers have examined code readability within the LLM context. Takerngsaksiri et al. [23] explored practitioners' perspectives on code readability in the age of LLMs through a case study, while Pan et al. [24] investigated the influence that readability has on the LLM budget when LLMs process code snippets. Furthermore, Hu et al. [25] investigated LLMs' performance when interpreting poor-readability code, finding that even LLMs struggle with low-readability code, which mirrors human developers' difficulties that lead to technical debt accumulation. These findings further emphasize the detrimental impact of low-readability code in software engineering.

C. Prompt Engineering

As an efficient alternative to costly tuning and retraining, prompt engineering has emerged as a primary paradigm for optimizing LLM performance across diverse tasks [26], [27]. Since prompt design directly shapes model outputs, it likely influences the readability of the generated code.

Leveraging their extensive pre-training, LLMs function as capable few-shot learners through zero-shot instructions or in-context examples [28], [29]. The adoption of prompt engineering across various domains has established task-specific best practices. Researchers have defined design principles for specialized contexts such as academic writing and business management [30], [31], while practical implementations like "cursor rules" [32] utilize pre-set contexts to optimize code generation. These frameworks improve LLMs' output by explicitly shaping the input prompt [30], [32], [33], including persona settings, stylistic guidelines, function signature, and technical contracts. Beyond structural optimization, prompt engineering also incorporates reasoning-oriented methods, as Chain-of-Thought (CoT) prompting [34] enhances performance by simulating step-by-step problem decomposition.

In summary, current research on LLM code generation primarily emphasizes functional capacity, and prompt engineering literature has prioritized strategies for enhancing functional performance, leaving the crucial non-functional dimension of readability unexplored. Evaluating the readability of LLM-generated code at scale, identifying specific issue patterns, and evaluating the impact of prompt design thus remain critical yet unaddressed knowledge gaps.

III. METHODOLOGY

As shown in Figure 1, we employed a mixed-method approach to address our research questions. We started by collecting and processing source data to obtain structured prompt and human implementation. Then we prompted LLMs to generate code responses, comparing the readability of

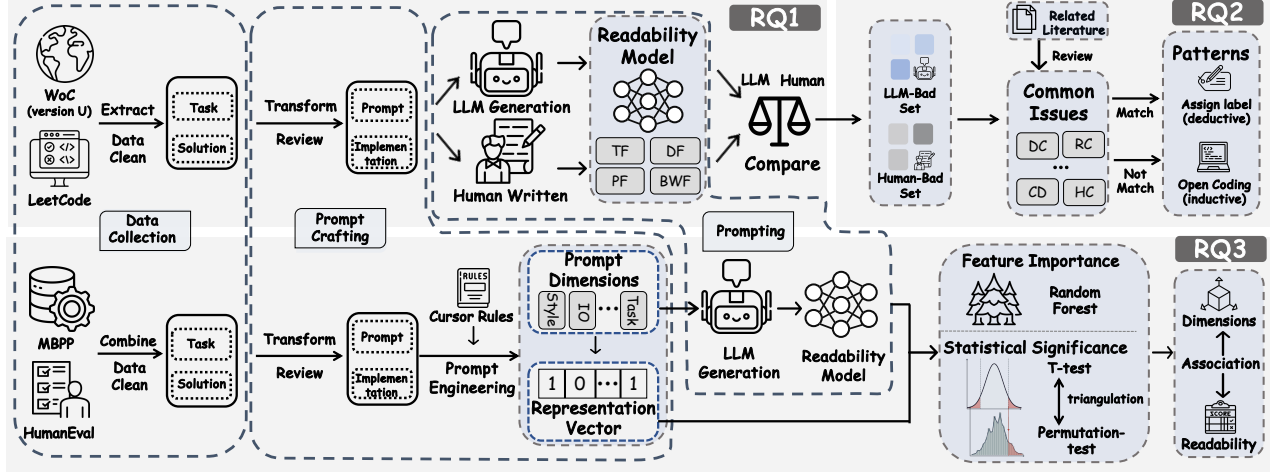


Fig. 1. Overview of the methodology

LLM-generated code with human-written code. After that, we identified readability issue patterns through hybrid deductive-inductive thematic analysis, and investigated the impact of different prompt dimensions on generated code readability using statistical methods.

A. Data Collection

To make LLM application scenarios more representable, we selected both real software engineering development contexts and competitive programming problems as task backgrounds. Specifically, we used World of Code (WoC) [9] and LeetCode [10] as data sources. Given that ChatGPT was released in November 2022 [35], before which LLMs were neither widely adopted nor integrated into programming workflows, we selected data snapshots prior to 2022 so that the collected code is human-written. Note that, although the pre-2022 data may overlap with the LLM’s pre-training corpus, this does not compromise the validity of our conclusions. LLMs acquire code generation capabilities through next-token prediction conditioned on statistical patterns learned from large-scale training corpora, and their outputs inherently reflect these learned distributions over human-written code. Whether or not a problem was encountered during pre-training, the generated code draws from the same underlying distribution, which is what practitioners obtain when using LLMs in practice. Unlike studies that assess functional correctness, where overlap with training data may artificially inflate pass rates on known benchmarks, our evaluation targets stylistic and structural properties of the generated code, which remain meaningful regardless of whether the solution logic has been previously encountered.

WoC is a comprehensive infrastructure for mining version control data across open-source software ecosystem, which aggregates Git objects, including commits, trees and blobs, on platforms like GitHub, Bitbucket, and GitLab [9], [36]. It provides mappings from project identification to project code, blobs, version control information, and other data for each project entity, facilitating easy querying and management.

We selected version U of WoC (released in October 2021) and used the `c2fbb` database, which maps each commit to

its corresponding commit file name, new blob (post-commit), and old blob (pre-commit), filtering for Python project files¹. To mitigate the risk of including non-representative code (e.g., toy projects or experimental repositories), we applied the following selection criteria: (1) we retained only projects with at least 10 commits, ensuring a minimum level of development activity and maturity; (2) we excluded projects whose repository names or descriptions contained keywords indicative of academic exercises or tutorials (e.g., “homework,” “assignment,” “tutorial,” “demo,” “test,” etc.); (3) following Buse and Weimer’s definition of readability [18], we utilized function-level snippets and retained only functions between 5 and 100 lines of code (excluding blank lines and comments). This granularity ensures sufficient context without spanning multiple methods, while filtering out trivially short snippets and excessively long functions that could introduce confounding complexity.

We therefore extracted and segmented Python source files into individual function-level entities for analysis. We further divided each snippet into a function header and body: the header, containing the function signature and docstring description, would serve as the prompt to stimulate LLM code generation, while the body, representing the concrete implementation, would serve as human-written baseline. Additionally, we excluded functions whose docstrings were absent or shorter than 10 words, as an insufficiently descriptive header would constitute an ambiguous prompt that cannot reliably guide LLM generation. After filtering and selection, we finally obtained 1,000 prompt-baseline pairs.

LeetCode [10] is an online platform that provides programming problems spanning various difficulty levels and topics. It offers a standardized problem set with clear problem descriptions in natural languages, and worldwide users provide a diverse array of human-written code in multiple programming languages [7].

We crawled all accessible problems from LeetCode, including their problem descriptions and code solutions, and filtered

¹To maintain a manageable research scope, in this study, we focused specifically on Python code readability.

for solutions implemented in Python and released before 2022. We then applied the following selection criteria: (1) for each problem, when multiple solutions existed, we selected the most popular one with the highest number of upvotes, as this metric reflects peer-validated quality and idiomatic style; (2) consistent with the WoC dataset, we retained only solutions whose core function body ranged between 5 and 100 lines of code. Subsequently, we extracted the core function implementation from each code solution to serve as the human-written baseline, with the corresponding problem description serving as the prompt.

Ultimately, we obtained 1,735 pairs from LeetCode. Combining with WoC-sourced data, we got 2,735 effective samples in total.

B. Prompt Crafting

To support our multi-stage investigation, we crafted two different prompt sets tailored to different research objectives. The **first set A** should provide a diverse array of real-world scenarios to evaluate the general readability of LLM-generated code and identify specific defect patterns (**RQ1 and RQ2**), while the **second set B** should facilitate a controlled experimental environment for **RQ3**. This strategic separation allows us to maintain ecological validity for problem identification while ensuring the empirical rigor required for variable-controlled analysis of prompt dimensions.

1) Set A: Real-World Scenario Prompts (RQ1 & RQ2):

This set establishes a baseline for general readability and defect identification. Specifically, we used the 2,735 samples previously collected from real software engineering practice as the foundational data, which includes 1,735 natural language problem descriptions from LeetCode and 1,000 function-level docstrings from WoC.

The LeetCode problem descriptions are already self-contained natural language task specifications and were used directly as prompts without modification, while the WoC docstrings are inline reference annotations for existing implementations (e.g., parameter type lists, return value notes) rather than as task-oriented instructions to prompt code generation. Directly using such docstrings as prompts would produce inputs unrepresentative of how developers interact with LLMs in practice, where users formulate requests in instructional natural language [37]. To address this, two authors independently transformed the WoC docstrings into natural language task descriptions following three constraints:

- **Semantic preservation:** the functional requirement expressed must be strictly equivalent to the original docstring with no addition or omission of logic;
- **Minimal stylistic intervention:** the transformation adopts plain instructional phrasing without embellishing with stylistic or readability-related descriptions;
- **Being self-contained:** the resulting prompt must be interpretable without access to the surrounding codebase.

This process involved independent transformation followed by a cross-validation phase in which each author reviewed the other’s outputs against the three constraints. Any discrepancies were resolved through iterative discussion to a consensus on

all final expressions. We finally got a comprehensive set of 5,086 prompts.

2) *Set B: Controlled Experimental Prompts (RQ3):* Analyzing how specific prompt dimensions affect readability requires a rigorous control of variables. The complexity of real software practice in the first set makes it difficult to decouple specific descriptive elements into isolated variables. Thus, we adopted a from-scratch construction strategy, prioritizing simplicity and modularity to ensure experimental control.

We sourced problems from MBPP [38] and HumanEval [8] as they are established benchmarks with concise descriptions that are easily decoupled into base components. To maintain a balanced sample size, we combined 164 HumanEval samples with a randomly sampled subset of 164 MBPP problems. Similarly, natural language task descriptions originally provided by MBPP were used as prompts without modification, while for HumanEval, whose prompts are formatted as embedded Python docstrings, we converted them into standalone natural language instructions following the same three constraints. This process was also followed by author cross-validation to ensure rigor and accuracy.

Referring to prior work [27], [30], [39], we limited the dimensions of prompt crafting to 8 types, as shown in Table I. *Minimum task description* served as the baseline, consisting of the most basic problem description without any additional information. *Function signature*, *IO contract*, *few-shot instruction*, and *task category* were extracted from the code implementation and test cases, while *style description*, *persona setting*, and *constraint* were derived from best practices in cursor rules [32]. Starting with each problem’s baseline prompt, we constructed prompts that only added one dimension, omitted only one dimension, or included all dimensions and recorded dimensions that current prompt contained with a seven-element vector:

$$V_{\text{prompt}} = (\mathbb{I}\{d\})_{d \in \mathcal{D}},$$

where $\mathcal{D} = \{\text{style, signature, IO, few-shot, category, persona, constraint}\}$, and $\mathbb{I}\{d\} = 1$ if dimension d is included in the prompt, otherwise 0.

Finally, each baseline corresponded to 16 different prompts and vectors representing their crafting elements, ultimately yielding 5,248 operational (*prompt, vector*) pairs in set B.

C. Prompting and Data Processing

For the prompt set A (RQ1 & RQ2), we input the 2,735 prompts into a set of prevalent LLMs, including GPT-5 [40], Claude-4.6 [41], DeepSeek-v4 [42], Gemini 3 [43] and Qwen 3.5 [44], to obtain their responses, with *temperature* set to 0 and *max_tokens* set to 4,096. We extracted code body from the markdown-formatted text returned by LLMs and cleaned non-function parts, such as execution examples. For the LeetCode subset, we verified functional correctness by executing each generated solution against the platform-provided test cases, achieving pass@1 of 95.6% (GPT-5), 94.5% (Claude-4.6), 95.6% (Deepseek-v4), 95.5% (Gemini 3), and 95.0% (Qwen 3.5). Solutions that failed any test case were excluded. For the WoC subset, no

executable test suites are available, as functions extracted from production repositories lack self-contained test. However, the consistently high pass rates on LeetCode, along with those observed on MBPP and HumanEval reported below, indicate that LLMs maintain reliable correctness on tasks of comparable complexity, providing no evidence of systematic degradation on WoC-sourced prompts. We then applied the readability model to LLM-generated code and the corresponding human-written baseline. After that, we compared the readability between code authored by LLMs and humans, as well as between code generated by different LLMs.

Regarding the prompt set B (RQ3), to conserve computing resources, we selected `claude-4.6` for further experimentation, as it demonstrated the best readability performance among functionally correct solutions (Section V). We then inputted the 5,248 prompts to obtain the returned markdown-formatted text (*temperature* = 0 and *max_tokens* = 4,096), and used similar methods to extract Python code and remove extraneous information. We executed the generated code against test cases provided, observing pass@1 of 95.1% (MBPP) and 96.3% (HumanEval), and retained those that passed all tests ($N = 5,024$). We then evaluated the readability of the generated code using our readability model.

Note that readability constitutes an independent dimension of code quality and remains observable and practically relevant during code review and debugging regardless of functional correctness. Moreover, we compared the readability scores between solutions that passed and failed the test cases using a Mann-Whitney U test [45] and found no significant difference with $p = 0.27$. Nevertheless, we still restricted our analysis to verified-correct code where feasible, ensuring that our findings more reflect code that practitioners would adopt in production.

D. Thematic Analysis

To address RQ2, we conducted a comparative thematic analysis [46], [47] of LLM-generated code and human-written code. We adopted a best-case sampling strategy because it provides a forward-looking perspective on readability issues that may become more relevant in practice, as LLMs continue to evolve and users have access to multiple generated alternatives. This design allows us to identify which readability issues persist even in the most readable outputs available in our experimental setting. Specifically, for each programming task in the prompt set, we selected the output with the highest readability score among evaluated LLMs as the representative LLM-generated snippet for that task. We then paired each representative LLM-generated snippet with the human-written counterpart and randomly sampled 500 code pairs² from the WoC-sourced and LeetCode-sourced datasets, respectively. Based on readability scores within each pair, samples were divided into cases where LLM-generated code scored higher than its human-written counterpart (405 WoC, 328 LeetCode) and cases where human-written code scored higher (95 WoC, 172 LeetCode). Two authors then independently examined and annotated the sampled pairs.

²The sample size of 500 per source was determined to balance analytical depth with the practical feasibility of manual annotation.

Our coding procedure followed a hybrid deductive-inductive approach. For each pair, the annotators first read both LLM-generated and human-written code in full and identified which of the four readability dimensions (detailed in Section IV) contributed to the lower readability of the less readable snippet: Textual Features (TF) [22], Posnett’s Features (PF) [19], Buse and Weimer’s Features (BWF) [18], and Dorn’s Features (DF) [48]. Note that, since a single snippet could exhibit readability issue on multiple dimensions, dimension-level coding was treated as a multi-label annotation task, allowing annotators to assign more than one readability dimension when appropriate. Subsequently, We constructed an initial deductive coding scheme by synthesizing nine common readability issue patterns from previous studies on code readability [20], [22], [49]–[51], as summarized in Table II. The annotators applied these predefined patterns to the less readable snippets in each pair. Similarly, this process was also treated as a multi-label annotation task.

When an observed readability issue could not be adequately captured by any of the nine predefined patterns, the annotators recorded it through open coding [52]. Each annotator independently assigned a descriptive label to the observed issue, and then two annotators compared their open codes. Labels referring to the same phenomenon were merged under a unified name iteratively through discussion, while labels proposed by only one annotator were reviewed jointly and either retained or discarded by consensus. The retained open codes were further synthesized into additional readability issue patterns distinct from the predefined ones.

To assess inter-rater reliability, we calculated Cohen’s Kappa values. Since our coding process allowed multiple labels per snippet, we evaluated agreement separately for each label by converting it into a binary coding decision. For readability-dimension coding, Cohen’s Kappa values across the four dimensions ranged from 0.81 to 0.89, with a mean of 0.84, while for issue-pattern coding, Cohen’s Kappa values across the predefined issue patterns ranged from 0.73 to 0.85, with a mean of 0.79, indicating substantial inter-rater agreement. All discrepancies were discussed and resolved by consensus to produce the final results.

E. Statistical Analysis

To address RQ3, we conducted a statistical analysis of the state vectors of 5,024 prompts in set B and the corresponding readability scores of generated code. First, we performed a random forest regression [53] using all prompt dimensions as predictors, with $n_estimators = 100$, $max_depth = 15$, and $min_samples_split = 5$. Random forest regression is appropriate for our analysis because it can model non-linear associations between prompt dimensions and readability scores, while accounting for interaction effects among dimensions. Using the model-based feature importance estimates [54], we identified dimensions with stronger predictive relevance to generated code readability.

We then evaluated the statistical significance of each dimension’s association with code readability through methodological triangulation, combining t-test [55] and permutation

TABLE I
DIMENSIONS OF PROMPT CRAFTING

Dimension	Explanation
<i>Minimum task description</i>	The essential instruction or objective of the prompt, stated in a brief and self-contained form.
<i>Style description</i>	Instructions specifying the desired code presentation style, such as formatting, naming, and commenting.
<i>Function signature</i>	An instruction specifying the expected function interface, typically including the function name, parameters, and type hints.
<i>IO Contract</i>	A precise description of the expected input and output structures, formats, and contents for the generated code.
<i>Few-shot instruction</i>	Providing the model with examples of the input/output pairs for the specific task.
<i>Task category</i>	Explicitly naming the general type of problem or task that the model is being asked to perform.
<i>Persona setting</i>	Assigning the model a specific identity, role, or expertise to adopt during generation.
<i>Constraints</i>	Restrictive guidelines specifying properties that the generated code should avoid or minimize.

importance test [56]. The t-tests were used to directly compare the readability scores of code generated under two prompt settings (i.e., baseline prompts vs. prompts with the tested dimension augmented) and to determine whether their score differences were statistically significant (when $p < 0.01$). Permutation importance tests assessed whether the tested dimension contributed to model-based prediction performance. Specifically, we used the random forest model as prediction kernel and randomly shuffled the binary indicator of the tested dimension over 10,000 iterations. A dimension was considered relevant if preserving its original indicator led to significantly better predictive performance than the shuffled cases (when $p < 0.01$). By synthesizing the evidence, we identified whether each dimension significantly correlates with generated code readability. To further ensure reliability, we conducted an ablation analysis comparing full-dimension prompts with prompts in which one dimension was removed, thereby examining whether the absence of each dimension produced convergent evidence in the opposite direction.

IV. READABILITY MODEL

To evaluate code readability, we constructed a readability assessment model from a broad pool of established readability indicators, followed by feature selection and model construction to derive the final assessment model. Specifically, we first organized candidate features from four commonly studied families of readability-related indicators: TF [22], PF [19], BWF [18], and DF [48]. This selection is based on extensive code readability literature [18], [19], [21], [22], [48], [57], representing widely adopted indicators of code readability across different programming contexts.

From this candidate feature space, we developed our readability model through feature selection, model training, and empirical validation, retaining indicators that were empirically informative for readability estimation. The resulting model integrates established readability indicators into a calibrated measurement tool for assessing code readability.

TABLE II
COMMON READABILITY ISSUE PATTERNS

Issue Pattern	Explanation
<i>Deficient Comment (DC)</i>	Complex or non-obvious code lacks necessary explanations of its purpose, behavior, or important implementation logic.
<i>Redundant Comment (RC)</i>	Comments merely duplicate information that is already clear from the code itself, offering no additional value.
<i>Inconsistent Style (IS)</i>	The code fails to maintain a consistent style or follow common conventions in indentation, formatting, or idiomatic usage.
<i>Excessive Complexity (EC)</i>	The logic or control flow is overly convoluted, deeply nested, or implemented via unnecessarily complicated mechanisms.
<i>Poor Structure (PS)</i>	Code snippets are poorly organized, with unclear responsibility separation or weak grouping of related operations.
<i>Poor Naming (PN)</i>	Names for variables, functions, classes, or other identifiers are vague, misleading, or inconsistent with their purpose or scope.
<i>Magic Values (MV)</i>	The code directly uses raw constant values (e.g., numbers or strings) without explanatory context.
<i>Code Duplication (CD)</i>	Highly similar code fragments appear multiple times within the snippet or program, increasing maintenance cost error risk.
<i>High Coupling (HC)</i>	Components, functions, or modules are excessively dependent on one another, so that a change in one component requires cascading changes in others.

In this section, we focus on the rationale for the candidate feature families and summarize key implementation process and evaluation results. The complete technical details and mathematical definitions are provided in the supplementary appendix.

A. Metric

1) *TF: Semantic and Lexical Perspective*: To capture semantic and lexical aspect of code readability, we incorporated TF features [21], [22]. This feature family is motivated by the observation that identifiers and comments are primary carriers of developer intent [58], [59]. Accordingly, TF characterizes whether the wording used in code is natural, semantically precise, and internally consistent with the documented behavior of the snippet. Representative signals in this family describe naming quality, comment-code consistency, lexical ambiguity, and conceptual cohesion. Collectively, these features reflect how easily a reader can relate program text to the underlying problem domain and recover the intended meaning of the code implementation.

2) *BWF: Structural Formatting Perspective*: We employed BWF features [18] to capture the formatting and layout characteristics. This family focuses on low-level structural regularities that can be perceived directly from the arrangement of code on the screen without deep semantic analysis. In our setting, BWF summarizes three complementary aspects: the horizontal density of individual lines, the syntactic distribution within lines, and the vertical segmentation induced by blank lines and comments. Collectively, these features reflect whether a snippet is visually crowded, unevenly distributed, or clearly partitioned into manageable units, which affect scanning effort and local comprehension [18], [60].

3) *PF: Information-Theoretic Perspective*: To assess readability from an information-theoretic perspective, we incorporated PF features based on entropy [19]. Code becomes more difficult to process when its information content is high or its token distribution is irregular. This family therefore captures the compactness, diversity, and unpredictability of the textual signal, measuring the overall information volume of a snippet and the degree of lexical disorder between lines. From a cognitive perspective, these measures approximate the amount of information that a reader must organize and retain when reading code [61], [62].

4) *DF: Visual and Geometric Perspective*: To account for visual and geometric properties, we adopted DF features [48], which model code as a visual object. This feature family characterizes the shape, alignment, and visual distribution of code on the page, reflecting whether a snippet exhibits a regular spatial layout and consistent alignment of syntactic elements. Such properties influence how efficiently readers can detect visual patterns before fully parsing the text, as spatial regularity and alignment facilitate code navigation [63]–[66].

Together, these four feature families offer complementary coverage of code readability, reducing the risk of relying on any single class of indicators.

B. Implementation and Evaluation

To operationalize the metric families described above, we implemented a unified feature extraction pipeline that transforms raw code snippets into snippet-level readability features [67]. We then instantiated the four feature families by implementing the corresponding indicators according to their definitions. All metrics are represented as scalar features at the snippet level and concatenated in a fixed order to form the raw feature vector $\mathbf{x} = [\text{TF}, \text{BWF}, \text{PF}, \text{DF}]$. Specifically, TF, BWF, PF, and DF yield 16, 26, 4, and 15 features, respectively, resulting in a 61-dimensional candidate feature space³.

To perform feature selection and evaluate our readability model, we conducted experiments on the Dorn dataset [48], a widely used benchmark for code readability evaluation consisting of 360 code snippets. Each snippet is associated with a binary readability label derived from human expert annotations.

We adopted a 10-fold stratified cross-validation protocol [68]. Within each fold, feature standardization was fitted on the training split and then applied to the test split to avoid data leakage. Given the size of the 61-dimensional candidate feature space, we applied Sequential Forward Selection (SFS) [69], [70] to identify a discriminative subset of features. We utilized a Logistic Regression classifier with L2 regularization as the base estimator for SFS. SFS was performed only within the training split of each fold, starting from an empty set and iteratively adding features that improved validation performance. Table III reports the optimal number of selected features for each setting.

³Full implementation details are recorded in the appendix, including language-specific extraction patterns, normalization procedures, and feature aggregation mechanisms.

TABLE III
CROSS-VALIDATED PERFORMANCE OF OUR READABILITY MODEL

Feature Family	Features	Accuracy	AUC
TF	16	64.2%	64.4%
TF+SFS	3	64.4%	68.5%
BWF	26	72.1%	78.7%
BWF+SFS	10	72.9%	80.5%
PF	4	66.7%	71.2%
PF+SFS	2	63.3%	69.3%
DF	15	66.1%	75.8%
DF+SFS	10	67.8%	76.6%
All-features	61	71.7%	80.5%
All-features+SFS (default)	21	74.4%	83.4%
All-features+SFS ($n_{\text{features}} = 25$)	25	77.5%	83.8%
All-features+LFS	14	72.8%	73.7%

TABLE IV
READABILITY SCORES OF LLM-GENERATED AND HUMAN-WRITTEN CODE

Model	Data Source	Avg (LLM)	Avg (Human)	Win Rate (%)
Claude-4.6	WoC	0.11	-0.15	55.00
	LeetCode	3.47	3.14	57.75
GPT-5	WoC	0.17	-0.15	55.70
	LeetCode	2.93	3.17	41.76
Gemini 3	WoC	0.53	-0.15	64.70
	LeetCode	3.27	3.14	50.29
DeepSeek-v4	WoC	0.31	-0.15	60.60
	LeetCode	3.21	3.17	49.50
Qwen 3.5	WoC	-0.05	-0.15	51.30
	LeetCode	3.29	3.14	55.10
All	WoC	0.22	-0.15	57.46
	LeetCode	3.25	3.15	51.15

* **Avg** refers to the average readability score of corresponding code snippets evaluated by the readability model; **Win Rate** refers to the proportion of samples where LLM-generated code strictly outperforms the human-written counterpart in readability.

* **Avg(Human)** scores on LeetCode vary slightly across models because instances where LLM-generated counterpart fails the test are excluded from the calculation.

We report Accuracy and Area Under the ROC Curve (AUC) [71], which capture classification accuracy and ranking quality, respectively. The evaluation covers the four individual feature families, their concatenation (All-features) and the comprehensive model developed by Linear Forward Selection (LFS) in previous research [22]. As shown in Table III, combining the four feature families provides better predictive power for readability over a single feature family. Moreover, feature selection improves the integrated representation, as All-features+SFS ($n_{\text{features}} = 25$) achieved the best overall result with 77.5% accuracy and 83.8% AUC. The SFS-based settings also performed better than All-features+LFS (72.8% accuracy and 73.7% AUC), indicating that SFS is more effective for selecting discriminative readability indicators.

These results indicate that the four metric families provide complementary information and that feature selection is an important step for transforming the raw integrated metrics into a more effective readability assessment model, leading to consistent improvements over individual feature families and previous standalone configurations.

V. RQ1: READABILITY COMPARISON

Table IV shows that LLM-generated code is generally comparable to, and in many settings slightly more readable

than, human-written code. In WoC, all models achieve higher average readability scores than the human-written counterparts, with win rates ranging from 51.30% to 64.70%. While in LeetCode, the advantage is weaker: several models show only marginal advantages, and GPT-5 falls below the human baseline in both the average score and win rate. Note that although WoC samples exhibit lower absolute scores due to more complex contexts inherent in real-world software tasks, our comparison remains within-task, with each LLM output compared against the corresponding human-written code.

This difference suggests that the relative readability of LLM-generated code depends on the task setting. LeetCode problems are usually self-contained and well-specified [72], and human programmers often have accumulated stable solution patterns for such tasks [73], leading to relatively concise and readable implementations and leaving limited room for LLM-generated code to improve. By contrast, WoC samples come from more complex real-world software contexts [9], where implementation details, dependencies, and functionality-first practices [73] may make human-written code less uniformly readable. LLMs may benefit from style regularities learned during pretraining [8], producing code that appears more standardized and readable under our assessment model. However, since WoC snippets are not accompanied by executable test suites, this readability advantage should not be interpreted as evidence of functional correctness.

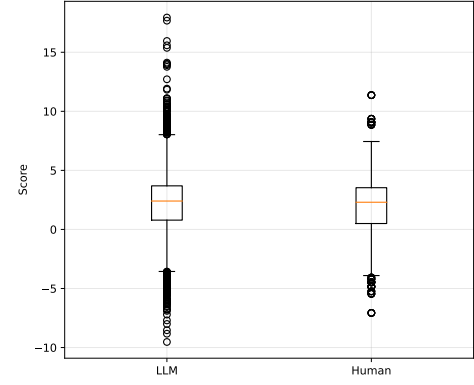
Figure 2 further illustrates the readability score distributions, revealing that while LLMs achieve a marginally higher median, human-written code demonstrates better consistency, as evidenced by a tighter interquartile range and fewer outliers (Figure 2a). This suggests that LLM-generated code shows more variation across tasks, which may be attributed to LLMs’ inherent next-token prediction mechanism [74], in contrast to programmers’ established programming habits [73]. Figure 2b (15 extreme outliers are excluded for better visualization) intuitively shows substantial distributional overlap between LLM-generated and human-written code.

To establish a more rigorous comparison, we applied the Wilcoxon signed-rank test [75] to all matched LLM-human pairs, thereby controlling for task-specific variance. The result reveals a statistically significant difference between two groups ($p < 0.001$), but the effect size is small ($r = 0.104$), especially given the large number of paired samples ($N = 13,263$). Thus, the statistical evidence supports a small but detectable difference rather than a substantial readability gap.

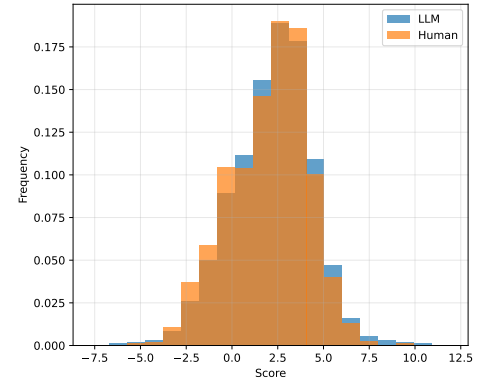
Taken together, these results suggest that LLM-generated code and human-written code are generally comparable in readability. The slight advantage observed for LLM-generated code is more evident in WoC than in LeetCode, indicating that readability differences vary across benchmark contexts with different task settings.

Beyond the overall LLM-human comparison, the 2,735 prompt-code pairs also provide a benchmark enabling a cross-model evaluation of code readability. As shown in Figure 3, Claude-4.6 achieves the strongest readability performance on functionally verified LeetCode code and maintains a stable overall readability profile across the evaluated settings. We therefore use Claude-4.6 in the experimental setting (RQ3,

Section III-C) as a strong representative model.

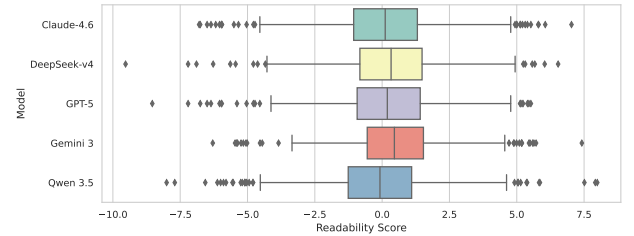


(a) Boxplot of the readability scores

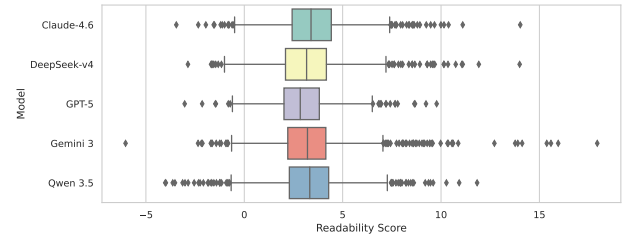


(b) Histogram of the readability scores

Fig. 2. Comparison of the readability scores of LLM-generated code and Human-written code



(a) WoC-sourced prompt results



(b) LeetCode-sourced prompt results

Fig. 3. Readability score distribution of code generated by different LLMs

VI. RQ2: READABILITY ISSUE PATTERNS

Table V shows that less readable human-written and LLM-generated code exhibit different dimension-level patterns.

Human-written snippets mainly suffer from BWF and TF issues, indicating problems in formatting, layout, comments, and textual expression, while PF is much less frequent. In contrast, less readable LLM-generated snippets are dominated by TF and PF issues, suggesting problems in semantic expression and unnecessary information load rather than surface formatting.

These distinct patterns reflect the underlying differences in code generation context. Human programmers often prioritize functional correctness and development efficiency over stylistic rigor and considerate textual expression, leading to BWF and TF issues [76], [77]. However, their experience-driven reliance on validated concise logic typically results in lower information entropy and predictable structures, thereby minimizing PF issues [73]. In contrast, having been trained on substantial amounts of standardized code [74], LLMs tend to generate style-standard code with less BWF issues, but their lack of experiential best practice often leads to correct but over-engineered solutions, manifesting as PF logic bloat [6], [78]. Furthermore, TF emerged as the most frequent issue dimension in LLM-generated code, which could be attributed to the lack of emphasis on semantic appropriateness in training data and model’s tendency to hallucinate text that is irrelevant to the actual code logic [79], [80]. These patterns indicate that LLM and human code can have similar readability scores for different underlying reasons.

The issue categories in Table VI make these dimension-level patterns more concrete, showing which observable coding problems give rise to the dimensional concerns in each group. In human-written code, the most frequent issue patterns are Deficient Comments (DC) and Inconsistent Style (IS), which align with the high TF and BWF counts in Table V, while Redundant Comments (RC) and High Coupling (HC) are rare. This pattern is consistent with the tendency of programmers to prioritize functional correctness and to speed over non-essential tasks like documentation and styling [77]. Regarding comments as secondary to execution, developers often omit necessary explanations or simplify them too much, which leads to high DC but very few RC issues [76].

The most frequent issue in LLM-generated code is Excessive Complexity (EC), followed by Redundant Comments (RC), which align with the high TF and PF counts in less readable LLM-generated snippets. Unlike humans, LLMs do not inherently aim for a simple solution. Instead, they may produce code with extra intermediate steps, over-verbose control flow, or unnecessary explanations, making the code harder to follow even when its formatting is clean and function is correct [6], [78], [80]. The low incidence of Code Duplication (CD) suggests that LLMs are good at modularizing code and reusing logic without repetition. High Coupling (HC) does not appear in either group, likely due to the function-level granularity of our snippets. We further discuss it in Section VIII.

Beyond predefined issue categories, open coding revealed three additional issues patterns in LLM-generated code: Unknown API, Redundant Variables, and Overblanking. Unknown API occurs when LLMs introduce external library calls without sufficient context, no matter whether these APIs are hallucinations or real. Even if the API is valid, its presence can make the snippet harder to understand when the dependency

TABLE V
DISTRIBUTION OF ISSUES ON READABILITY DIMENSIONS

Category	TF	BWF	PF	DF
Human-bad	72	76	<u>21</u>	45
LLM-bad	60	26	<u>53</u>	30
Total	132	102	<u>74</u>	75

* TF (Semantic/Lexical) evaluates intent and naming; BWF (Structural/Formatting) assesses layout and density; PF (Information-Theoretic) measures logical complexity and entropy; DF (Visual/Geometric) captures spatial alignment.

TABLE VI
CATEGORIES OF READABILITY ISSUE PATTERNS

Category	DC	RC	IS	EC	PS	PN	MV	CD	HC
Human-bad	60	3	58	41	24	22	7	6	0
LLM-bad	10	24	15	48	16	21	8	<u>2</u>	0
Total	70	27	73	89	40	43	15	<u>8</u>	0

is unfamiliar or its role is not explained [81], [82]. Redundant Variables refer to cases where multiple variables serve the same functional purpose, creating unnecessary indirection and making the data flow harder to trace. Overblanking involves excessive but meaningless blank lines that lengthen the code without improving structural separation. Although related to formatting, we treat it separately from IS and PS because it reflects a different problem, over-segmentation without a semantic boundary, instead of stylistic inconsistency or flawed logical organization. Its repeated occurrence in LLM-generated code suggests a surface-level use of spacing that does not always align with the logical structure.

In summary, LLM-generated and human-written code exhibit distinct readability failure modes: human-written code more often lacks explanation and consistent presentation, while LLM-generated code more often suffers from redundant explanation and unnecessary complexity.

VII. RQ3: ROLE OF PROMPT DESIGN

Figure 4 reports the feature importance scores of the random forest regression, providing an overall ranking of prompt dimensions by their predictive relevance to generated code readability. *Function signature* shows the highest importance among all evaluated dimensions, followed by *constraints* and *style description*, while *task category* and *IO contract* contribute the least. This suggests that prompt components closer to the form of the generated code are more associated with readability. *Function signatures* define the interface, naming context, and expected structure of a function-level snippet [83], while *style descriptions* and *constraints* provide explicit guidance, offering “*dos and don’ts*”. In contrast, *task category* and *IO contract* mainly specify what problem to solve and how inputs and outputs should behave, which are more directly related to functional logic instead of readability.

The R^2 of the random forest model (< 0.3) further contextualizes this ranking. Prompt dimensions explain only part of the readability variation, while other factors, such as problem complexity, model capability, and generation variability, may also play substantial roles [78], [84]. We therefore interpret prompt dimensions as lightweight levers for improving readability, rather than as dominant determinants.

TABLE VII
P-VALUE OF STATISTIC ANALYSIS OF PROMPT DIMENSIONS AND CODE READABILITY

Dimension	Incremental		Ablation	
	T-test	Permutation	T-test	Permutation
<i>Style description</i>	0.001	0.001	0.795	0.850
<i>Function signature</i>	0.239	0.240	0.003	0.003
<i>IO Contract</i>	0.353	0.356	0.855	0.887
<i>Few-shot</i>	0.257	0.291	0.643	0.678
<i>Task category</i>	0.282	0.322	0.673	0.684
<i>Persona setting</i>	0.287	0.282	0.281	0.287
<i>Constraint</i>	0.001	0.000	0.807	0.960

* Bold values indicate statistical significance ($p < 0.01$).

Table VII further shows which prompt dimensions show statistically significant associations with code readability. In the incremental setting, where each dimension is added to a baseline prompt, *style descriptions* and *constraints* show significant positive associations with code readability. This suggests that when starting from a minimal task description, explicitly declaring readability-oriented guidance is useful, as *style descriptions* specify the desired form of readable code while *constraints* restrict undesirable forms.

The ablation setting provides a complementary perspective. When one dimension is removed from comprehensive prompts, only removing *function signature* leads to a significant change in code readability. This suggests that, in well-designed prompts, *function signature* plays a distinctive role in shaping code readability. Beyond merely adding guidance, signatures provide a structural anchor for generation by specifying the function interface, naming context, and expected organization of the snippet [83]. When other prompt dimensions are already present, removing this anchor has a clearer readability cost than removing other dimensions.

In summary, the analyses show a consistent but bounded role of prompt design, with *function signatures*, *constraints*, and *style descriptions* emerging as the most relevant prompt dimensions associated with code readability. *Style description* and *constraints* are most useful when the prompt is underspecified, whereas *function signature* is most important when the prompt is otherwise complete. These

dimensions offer practical targets for low-cost readability improvements, but the modest explanatory power of the random forest model cautions against treating prompt engineering as a complete solution.

VIII. DISCUSSION

A. Multi-Turn Interactions and Readability

Our study adopts a single-turn prompting setting, which does not fully capture the iterative, multi-turn interactions increasingly common in practical workflows. However, single-turn readability remains important in such settings, and we discuss the relevance of our findings to multi-turn readability refinement.

Readability as a human-perceived intermediate property. Unlike functional correctness, which can be checked automatically through test execution, readability is tied to human cognitive effort. In a multi-turn interaction, developers typically need to read the generated code at each turn before they can formulate meaningful refinement instructions. Thus, a multi-turn session can be regarded as a sequence of intermediate reading episodes. The comprehension cost incurred at each intermediate turn accumulates throughout the interaction and is not offset by an improved final output. Therefore, single-turn readability characterizes not merely an isolated snapshot but a recurring cost that developers may face at every iteration of a multi-turn workflow.

Illustrative example: multi-turn refinement of readability. To illustrate how multi-turn interaction may affect readability, we present an example in which we issue explicit follow-up instructions requesting readability improvements. Figure 5 suggests that targeted follow-up instructions can improve readability to some extent. However, the second-turn instruction is not independent of the first-turn code, which is formulated after inspecting the initial output and identifying what needs to be improved. This reinforces our preceding argument that such instructions are usually informed by the developer’s review of the preceding output, with reading cost already incurred.

Thus, single-turn readability remains a key prerequisite for effective multi-turn interaction. This also suggests a potential direction for future work to combine automated detection of readability issues with the generation of refinement instructions, so that the comprehension burden currently placed on developers can be reduced.

B. Implications

Our findings show that readability deserves attention in LLM-generated code, as it affects how generated code is evaluated, reviewed, and refined.

Extending model evaluation beyond correctness. Our study shows that LLM-generated code indeed present readability issues. Current benchmarks for code-generating LLMs mainly focus on functional correctness and task completion rate through unit tests [85], [86]. However, passing tests does not guarantee that generated code is easy to read, review, or maintain. Models that generate functionally correct but cryptic code can still impose substantial cognitive burdens

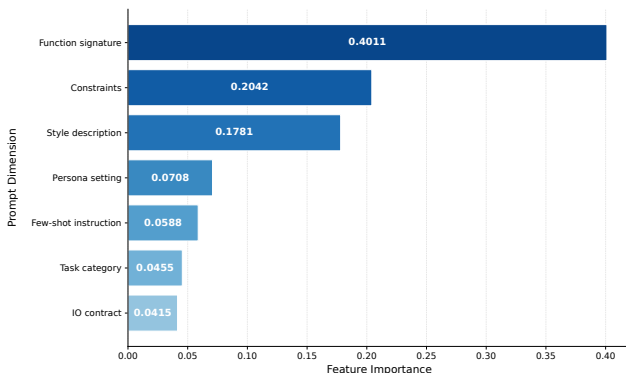


Fig. 4. Feature importance of the Random Forest regression result

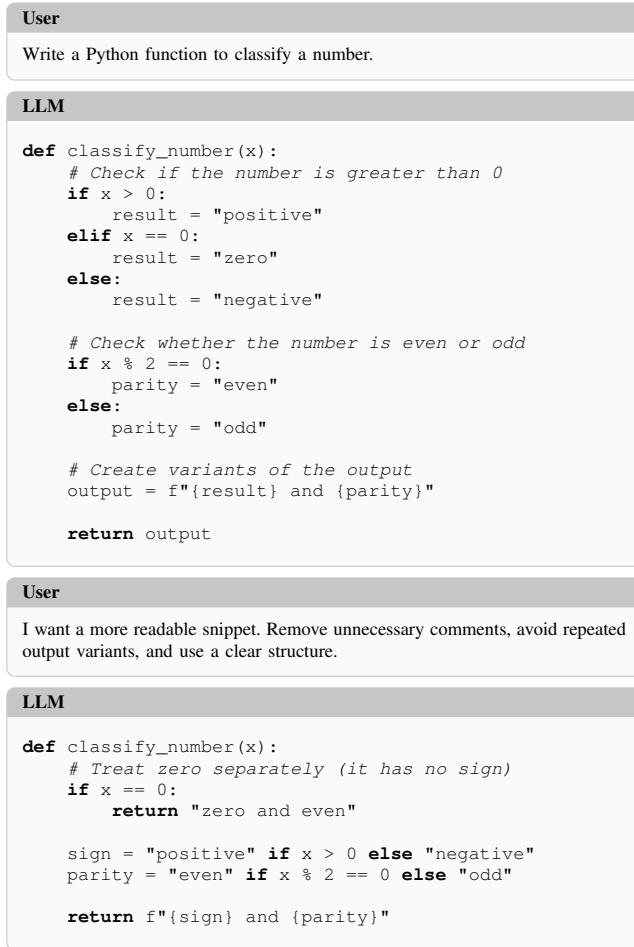


Fig. 5. An example of multi-turn refinement process in code generation

on developers during review and maintenance [87]. Future benchmarks should therefore evaluate generated code not only by whether it passes tests, but also by whether it supports human comprehension. For model developers, incorporating readability into evaluation may also support training and alignment procedures that favor maintainable code beyond merely executable code.

Managing readability debt in code review. Existing review and static analysis tools were largely designed around human-written code [88], but this work reveals that LLM-generated code presents different readability risks. Human readability issues often involve neglect, such as *Deficient Comments* and *Inconsistent Style*, whereas LLM-generated code more often involves overproduction, such as *Redundant Comments* and *Excessive Complexity*. These issues can accumulate into what we call readability debt: code that looks acceptable at generation time but increases future comprehension and maintenance cost. Code review tools should therefore include checks tailored to LLM-typical problems, detecting logical bloat, unnecessary variable redundancies, unexplained API usage, etc. Integrating these checks into CI/CD workflows can help prevent readability debt from entering the codebase while preserving the productivity benefits of AI-assisted coding.

Prompt design as a lightweight intervention. Prompt design offers a low-cost way to improve readability. Our

results suggest that *function signature*, *style description*, and *constraints* are the most actionable dimensions. *Style description* and *constraints* help underspecified prompts, while *function signature* matters when prompts are otherwise complete. Practitioners can use these dimensions as a starting point for readability-oriented prompting. However, since prompt design explains only part of readability variation, stronger improvements may require alternative approaches, such as post-processing [89], readability-oriented fine-tuning [28], or hybrid human-AI collaborative coding workflows [90].

Tool support for readability refinement. In multi-turn AI-assisted development, developers often need to read generated code before they can write useful refinement instructions [91]. This makes readability an interaction cost, not only a final-output property. We therefore advocate future tools that detect readability issues in generated snippets and produce in-context refinement suggestions based on the detected problems. For example, if a snippet contains excessive logic or unexplained API usage, the tool could suggest targeted follow-up prompts to simplify the control flow or clarify the role of API. Such support could reduce the diagnostic burden on developers while keeping them in control of final code quality.

C. Threats to Validity

Internal Validity. A key threat concerns the assessment of code readability. Although our readability model integrates four complementary feature families, applies feature selection, and achieves the best performance in the evaluation, readability remains subjective and multifaceted. The model may not fully capture individual developer preferences or project-specific readability norms. Furthermore, our focus on function-level snippets limits the detection of issues such as *High Coupling*, which typically manifests at the file or project level. So its absence in our findings should be interpreted as a result of our analysis granularity rather than evidence that such issues do not occur in LLM-generated code. Finally, for WoC and HumanEval, we converted docstrings into natural language task descriptions. Although this transformation followed strict constraints and underwent cross-validation, it may still have subtly affected the LLM outputs. Future work could compare outputs generated from original docstrings and transformed prompts to quantify this effect.

External Validity. One threat concerns our sampling strategy for analyzing readability issue patterns and the role of prompt design. For issue patterns, we selected the most readable LLM output among the evaluated models for each task, while for prompt design, we focused on Claude-4.6, the best model on functionally verified LeetCode code in our comparison. Thus, the corresponding findings should not be interpreted as describing average LLM-generated code. Rather, they characterize readability issues and prompt design associations under a strong LLM-output setting. This design may overestimate LLM readability, but it is appropriate for studying the residual readability weaknesses of strong models as LLM capabilities continue to evolve.

Another threat concerns the scope of our experimental setting. Our study focuses on Python and a selected set of LLMs,

and the findings may differ for programming languages with different syntactic and stylistic conventions [92] or for models specifically tuned for code generation. In addition, although our sample includes diverse sources, independent snippets cannot fully capture production environments, where code generation is shaped by project-specific constraints, broader dependencies, and collaborative workflows. Future work is expected to examine readability in larger, project-level human-AI development scenarios.

IX. CONCLUSION

This study examined the readability of LLM-generated code, its issue patterns, and the role of prompt design. Our results show that LLM-generated code is generally comparable to human-written code in overall readability, but the two differ in how readability breaks down. Human-written code more often suffers from *Deficient Comments* and *Inconsistent Style*, whereas LLM-generated code more often exhibits *Excessive Complexity*, *Redundant Comments*, and additional LLM-specific issues such as *Unknown API usage*, *Redundant Variables*, and *Overblanking*. We also find that prompt design can shape readability, with *function signature*, *style description*, and *constraints* being the most relevant dimensions. However, its overall role remains bounded, suggesting that prompt engineering is useful but incomplete. These findings highlight the need to evaluate LLM-generated code not only by functional correctness, but also by the concrete readability issues that may accumulate into latent technical debt. They also suggest that prompt design can serve as a lightweight starting point for readability improvement, while motivating future automated support for detecting, reviewing, and refining readability problems in AI-assisted development.

REFERENCES

- [1] J. L. Elshoff and M. Marcotty, "Improving computer program readability to aid modification," *Commun. ACM*, vol. 25, no. 8, pp. 512–521, 1982. 10.1145/358589.358596
- [2] D. Boswell and T. Foucher, *The art of readable code*. O'Reilly Media, Inc., 2011.
- [3] G. Digkas, A. Chatzigeorgiou, A. Ampatzoglou, and P. Avgeriou, "Can clean new code reduce technical debt density?" *IEEE Trans. Softw. Eng.*, vol. 48, no. 5, pp. 1705–1721, 2022. 10.1109/TSE.2020.3032557
- [4] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, pp. 21558–21572. 10.52202/075280-0943
- [5] L. Zhong and Z. Wang, "Can llm replace stack overflow? a study on robustness and reliability of large language model code generation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(19), 2024, pp. 21841–21849. 10.1609/aaai.v38i19.30185
- [6] Y. Zi, L. Li, A. Guha, C. Anderson, and M. Q. Feldman, "'i would have written my code differently': Beginners struggle to understand llm-generated code," in *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, 2025, pp. 1479–1488. 10.1145/3696630.3731663
- [7] T. Coignion, C. Quinton, and R. Rouvoy, "A performance study of llm-generated code on leetcode," in *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*, 2024, pp. 79–89. 10.1145/3661167.3661221
- [8] M. Chen, "Evaluating large language models trained on code," arXiv preprint *arXiv*, 2021.
- [9] Y. Ma, C. Bogart, S. Amreen, R. Zaretzki, and A. Mockus, "World of code: an infrastructure for mining the universe of open source vcs data," in 2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR). IEEE, 2019, pp. 143–154. 10.1109/MSR.2019.00031
- [10] "Leetcode," <https://leetcode.com/problemset/>.
- [11] H. Jin, L. Huang, H. Cai, J. Yan, B. Li, and H. Chen, "From llms to llm-based agents for software engineering: A survey of current, challenges and future," arXiv preprint *arXiv*, 2024.
- [12] R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, et al., "StarCoder: may the source be with you!" arXiv preprint *arXiv*, 2023.
- [13] Github, "Copilot," <https://github.com/features/copilot>.
- [14] Anysphere, "Cursor," <https://www.cursor.com/>.
- [15] D. Ramel, "Github copilot ai tech upgraded, already generates 61% of java code," 2023, <https://visualstudiomagazine.com/articles/2023/02/15/copilot-upgrade.aspx>.
- [16] Y. Yao, J. Duan, K. Xu, Y. Cai, Z. Sun, and Y. Zhang, "A survey on large language model (llm) security and privacy: The good, the bad, and the ugly," *High Confid. Comput.*, vol. 4, no. 2, p. 100211, 2024. 10.1016/j.hcc.2024.100211
- [17] W. Xu, K. Gao, H. He, and M. Zhou, "Licoeval: Evaluating llms on license compliance in code generation," in 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE). IEEE, 2025, pp. 1665–1677. 10.1109/ICSE55347.2025.00052
- [18] R. P. Buse and W. R. Weimer, "A metric for software readability," in *Proceedings of the 2008 international symposium on Software testing and analysis*, 2008, pp. 121–130. 10.1145/1390630.1390647
- [19] D. Posnett, A. Hindle, and P. Devanbu, "A simpler model of software readability," in *Proceedings of the 8th working conference on mining software repositories*, 2011, pp. 73–82. 10.1145/1985441.1985454
- [20] J. Johnson, S. Lubo, N. Yedla, J. Aponte, and B. Sharif, "An empirical study assessing source code readability in comprehension," in 2019 IEEE International conference on software maintenance and evolution (ICSME). IEEE, 2019, pp. 513–523. 10.1109/ICSME.2019.00085
- [21] S. Scalabrino, M. Linares-Vasquez, D. Poshyvanyk, and R. Oliveto, "Improving code readability models with textual features," in 2016 IEEE 24th International Conference on Program Comprehension (ICPC). IEEE, 2016, pp. 1–10. 10.1109/ICPC.2016.7503707
- [22] S. Scalabrino, M. Linares-Vasquez, R. Oliveto, and D. Poshyvanyk, "A comprehensive model for code readability," *J. Softw. (Malden)*, vol. 30, no. 6, p. e1958, 2018. 10.1002/smr.1958
- [23] W. Takerngsaksiri, C. Tantithamthavorn, M. Fu, J. Pasuksmit, K. Chen, and M. Wu, "Code readability in the age of large language models: An industrial case study from atlassian," in 2025 IEEE International Conference on Software Maintenance and Evolution (ICSME). IEEE, 2025, pp. 732–742. 10.1109/ICSME64153.2025.00073
- [24] D. Pan, Z. Sun, C. Zhang, D. Lo, and X. Du, "The hidden cost of readability: How code formatting silently consumes your llm budget," arXiv preprint *arXiv*, 2025.
- [25] C. Hu, Y. Chai, H. Zhou, F. Meng, J. Zhou, and X. Gu, "How effectively do code language models understand poor-readability code?" in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 795–806. 10.1145/3691620.3695072
- [26] G. Marvin, N. Hellen, D. Jjingo, and J. Nakatumba-Nabende, "Prompt engineering in large language models," in *International conference on data intelligence and cognitive informatics*. Springer, 2023, pp. 387–402.
- [27] J. White, "A prompt pattern catalog to enhance prompt engineering with chatgpt," arXiv preprint *arXiv*, 2023.
- [28] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, et al., "Finetuned language models are zero-shot learners," arXiv preprint *arXiv*, 2021.
- [29] E. Perez, D. Kiela, and K. Cho, "True few-shot learning with language models," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 34, 2021, pp. 11054–11070. <https://dl.acm.org/doi/10.5555/3540261.3541106>
- [30] L. Giray, "Prompt engineering with chatgpt: A guide for academic writers," *Ann. Biomed. Eng.*, vol. 51, no. 12, pp. 2629–2633, Dec. 2023. 10.1007/s10439-023-03272-4
- [31] K. Busch, A. Rochlitz, D. Sola, and H. Leopold, "Just tell me: Prompt engineering in business process management," in *International Conference on Business Process Modeling, Development and Support*. Springer, 2023, pp. 3–11. 10.1007/978-3-031-34241-7_1
- [32] Ashley, "What is. cursorrule and how to use it effectively," 2024, <https://medium.com/towards-agi/what-are-cursor-rules-and-how-to-use-the-m-ec558468d139>.
- [33] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, "Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing," *ACM Comput. Surv.*, vol. 55, no. 9, pp. 1–35, 2023. 10.1145/3560815
- [34] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, et al., "Chain-of-thought prompting elicits reasoning in large language

- models,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, pp. 24824–24837, 2022. 10.52202/068431-1800
- [35] OpenAI, “Chatgpt release,” <https://openai.com/index/chatgpt/>.
- [36] Y. Ma, T. Dey, C. Bogart, S. Amreen, M. Valiev, A. Tutko, et al., “World of code: Enabling a research workflow for mining and analyzing the universe of open source vcs data,” *Empir. Softw. Eng.*, vol. 26, no. 2, pp. 22–42, 2021. 10.1007/s10664-020-09905-9
- [37] V. Chen, A. Talwalkar, R. Brennan, and G. Neubig, “Code with me or for me? how increasing ai automation transforms developer workflows,” in *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*, 2026, pp. 1–19. 10.1145/3772318.3790850
- [38] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, et al., “Program synthesis with large language models,” arXiv preprint arXiv, 2021.
- [39] X. Zhang, J. Cao, C. You, and D. Ding, “Why prompt design matters and works: A complexity analysis of prompt search space in llms,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 32525–32555. 10.18653/v1/2025.acl-long.1562
- [40] A. Singh, A. Fry, A. Perelman, A. Tart, A. Ganesh, A. El-Kishky, et al., “Openai gpt-5 system card,” arXiv preprint arXiv, 2025.
- [41] Anthropic, “Introducing claude sonnet 4.6,” 2026, <https://www.anthropic.com/news/claude-sonnet-4-6>.
- [42] DeepSeek, “Deepseek v4 preview release,” 2026, <https://api-docs.deepseek.com/news/news260424>.
- [43] Google, “A new era of intelligence with gemini 3,” 2025, <https://blog.google/products-and-platforms/products/gemini/gemini-3/>.
- [44] QwenTeam, “Qwen3.5: Towards native multimodal agents,” 2026, <https://qwen.ai/blog?id=qwen3.5>.
- [45] N. Nachar, et al., “The mann-whitney u: A test for assessing whether two independent samples come from the same distribution,” *Tutor. Quant. Methods Psychol.*, vol. 4, no. 1, pp. 13–20, 2008. 10.20982/tqmp.04.1.p013
- [46] G. Terry, N. Hayfield, V. Clarke, and V. Braun, “Thematic analysis,” in *The SAGE Handbook of Qualitative Research in Psychology*, 2nd ed. London, U.K.: SAGE Publications, 2017, pp. 17–36. 10.4135/9781526405555.n2.
- [47] M. Williams and T. Moser, “The art of coding and thematic exploration in qualitative research,” *Int. Manag. Rev.*, vol. 15, no. 1, pp. 45–55, 2019.
- [48] J. Dorn, *A general software readability model*, vol. 5. MCS Thesis, 2012, pp. 11–14, available at web.eecs.umich.edu/~weimerw/students/dorn-mcs-paper.pdf.
- [49] V. Piantadosi, F. Fierro, S. Scalabrino, A. Serebrenik, and R. Oliveto, “How does code readability change during software evolution?” *Empir. Softw. Eng.*, vol. 25, no. 6, pp. 5374–5412, 2020. 10.1007/s10664-020-09886-9
- [50] F. Palomba, G. Bavota, M. Di Penta, R. Oliveto, and A. De Lucia, “Do they really smell bad? a study on developers’ perception of bad code smells,” in 2014 IEEE International conference on software maintenance and evolution. IEEE, 2014, pp. 101–110. 10.1109/ICSME.2014.32
- [51] D. Oliveira, R. Santos, B. De Oliveira, M. Monperrus, F. Castor, and F. Madeiral, “Understanding code understandability improvements in code reviews,” in *IEEE Transactions on Software Engineering*, vol. 51, no. 1, pp. 14–37, Jan. 2025, doi: 10.1109/TSE.2024.3453783.
- [52] J. Corbin and A. Strauss, *Basics of qualitative research: Techniques and procedures for developing grounded theory*. Sage publications, 2014.
- [53] L. Breiman, “Random forests,” *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001. 10.1023/A:1010933404324
- [54] R. Iranzad and X. Liu, “A review of random forest-based feature selection methods for data science education and applications,” *Int. J. Data Sci. Anal.*, vol. 20, no. 2, pp. 197–211, 2025. 10.1007/s41060-024-00509-w
- [55] P. Mishra, U. Singh, C. M. Pandey, P. Mishra, and G. Pandey, “Application of student’s t-test, analysis of variance, and covariance,” *Ann. Card. Anaesth.*, vol. 22, no. 4, pp. 407–411, Oct-Dec 2019. 10.4103/aca.ACA_94_19
- [56] A. Altmann, L. Toloşi, O. Sander, and T. Lengauer, “Permutation importance: A corrected feature importance measure,” *Bioinformatics*, vol. 26, no. 10, pp. 1340–1347, May 15 2010. 10.1093/bioinformatics/btq134
- [57] D. Lawrie, C. Morrell, H. Feild, and D. Binkley, “What’s in a name? a study of identifiers,” in 14th IEEE international conference on program comprehension (ICPC’06). IEEE, 2006, pp. 3–12.
- [58] D. Lawrie, H. Feild, and D. Binkley, “Syntactic identifier conciseness and consistency,” in 2006 Sixth IEEE International Workshop on Source Code Analysis and Manipulation. IEEE, 2006, pp. 139–148. 10.1109/S-CAM.2006.31
- [59] D. Lawrie, C. Morrell, H. Feild, and D. Binkley, “Effective identifier names for comprehension and memory,” *Innov. Syst. Softw. Eng.*, vol. 3, no. 4, pp. 303–318, 2007. 10.1007/s11334-007-0031-2
- [60] S. N. Woodfield, H. E. Dunsmore, and V. Y. Shen, “The effect of modularization and comments on program comprehension,” in *Proceedings of the 5th international conference on Software engineering*, 1981, pp. 215–223.
- [61] C. E. Shannon, “A mathematical theory of communication,” *Bell Syst. Tech. J.*, vol. 27, no. 3, pp. 379–423, 1948. 10.1002/j.1538-7305.1948.tb01338.x
- [62] M. H. Halstead, *Elements of Software Science, (Operating and programming systems series)*. Elsevier Science Inc., 1977.
- [63] M. Shamy and D. G. Feitelson, “Identifying lines and interpreting vertical jumps in eye tracking studies of reading text and code,” *ACM Trans. Appl. Percept.*, vol. 20, no. 2, pp. 1–20, 2023. 10.1145/3579357
- [64] P. R. F. de Oliveira, R. Gheyi, J. A. S. da Costa, and M. Ribeiro, “Assessing python style guides: An eye-tracking study with novice developers,” in *Simpósio Brasileiro de Engenharia de Software (SBES)*. SBC, 2024, pp. 136–146.
- [65] T. Busjahn, R. Bednarik, A. Begel, M. Crosby, J. H. Paterson, C. Schulte, et al., “Eye movements in code reading: Relaxing the linear order,” in 2015 IEEE 23rd international conference on program comprehension. IEEE, 2015, pp. 255–265. 10.1109/ICPC.2015.36
- [66] M. E. Crosby and J. Stelovsky, “How do we read algorithms? a case study,” *Computer*, vol. 23, no. 1, pp. 25–35, 1990. 10.1109/2.48797
- [67] L. Moonen, “Generating robust parsers using island grammars,” in *Proceedings eighth working conference on reverse engineering*. IEEE, 2001, pp. 13–22. 10.1109/WCRE.2001.957806
- [68] R. Kohavi et al., “A study of cross-validation and bootstrap for accuracy estimation and model selection,” in *Ijcai*, vol. 14, no. 2. Montreal, Canada, 1995, pp. 1137–1145.
- [69] I. Guyon and A. Elisseeff, “An introduction to variable and feature selection,” *J. Mach. Learn. Res.*, vol. 3, Mar, pp. 1157–1182, 2003.
- [70] A. W. Whitney, “A direct method of nonparametric measurement selection,” *IEEE Trans. Comput.*, vol. C-20, no. 9, pp. 1100–1103, 1971. 10.1109/T-C.1971.223410
- [71] T. Fawcett, “An introduction to roc analysis,” *Pattern Recognit. Lett.*, vol. 27, no. 8, pp. 861–874, 2006. 10.1016/j.patrec.2005.10.010
- [72] Y. Xia, W. Shen, Y. Wang, J. K. Liu, H. Sun, S. Wu, et al., “Leetcode-dataset: A temporal dataset for robust evaluation and efficient training of code llms,” arXiv preprint arXiv, 2025.
- [73] O. Meerbaum-Salant, M. Armoni, and M. Ben-Ari, “Habits of programming in scratch,” in *Proceedings of the 16th annual joint conference on Innovation and technology in computer science education*, 2011, pp. 168–172.
- [74] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, et al., “Attention is all you need,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.
- [75] F. Wilcoxon, “Individual comparisons by ranking methods,” *Biom. Bull.*, vol. 1, no. 6, pp. 80–83, 1945. 10.2307/3001968
- [76] RR de Almeida, C. Treude, and U. Kulesza, “What’s behind tight deadlines? business causes of technical debt,” in *Proceedings of the 16th International Conference on Cooperative and Human Aspects of Software Engineering*, Melbourne, Australia, 2023, pp. 14–15.
- [77] V. Lenarduzzi, T. Besker, D. Taibi, A. Martini, and F. A. Fontana, “A systematic literature review on technical debt prioritization: Strategies, processes, factors, and tools,” *J. Syst. Softw.*, vol. 171, p. 110827, 2021. 10.1016/j.jss.2020.110827
- [78] S. A. Licorish, A. Bajpai, C. Arora, F. Wang, and K. Tantithamthavorn, “Comparing human and llm generated code: The jury is still out!” arXiv preprint arXiv, 2025.
- [79] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, et al., “Survey of hallucination in natural language generation,” *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023. 10.1145/3571730
- [80] S. Kang, L. Milliken, and S. Yoo, “Identifying inaccurate descriptions in llm-generated code comments via test execution,” arXiv preprint arXiv, 2024.
- [81] M. P. Robillard and R. DeLine, “A field study of api learning obstacles,” *Empir. Softw. Eng.*, vol. 16, no. 6, pp. 703–732, 2011. 10.1007/s10664-010-9150-8
- [82] C. Wang, K. Huang, J. Zhang, Y. Feng, L. Zhang, Y. Liu, et al., “Llms meet library evolution: Evaluating deprecated api usage in llm-based code completion,” arXiv preprint arXiv, 2024.
- [83] X. Ding, R. Peng, X. Chen, Y. Huang, J. Bian, and Z. Zheng, “Do code summarization models process too much information? function signature may be all that is needed,” *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 6, pp. 1–35, 2024. 10.1145/3652156

- [84] D. Alawad, M. Panta, M. Zibran, and M. R. Islam, “An empirical study of the relationships between code readability and software complexity,” arXiv preprint *arXiv*., 2019.
- [85] Y. Wang, H. Ivison, P. Dasigi, J. Hessel, T. Khot, K. R. Chandu, et al., “How far can camels go? exploring the state of instruction tuning on open resources,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023, pp. 74764–74786. 10.52202/075280-3268
- [86] S. Lu, D. Guo, S. Ren, J. Huang, A. Svyatkovskiy, A. Blanco, et al., “Codexglue: A machine learning benchmark dataset for code understanding and generation,” arXiv preprint *arXiv*., 2021.
- [87] C. Jaspan and C. Green, “Defining, measuring, and managing technical debt,” *IEEE Softw.*, vol. 40, no. 3, pp. 15–19, 2023. 10.1109/MS.2023.3242137
- [88] S. Panichella, V. Arnaoudova, M. Di Penta, and G. Antoniol, “Would static analysis tools help developers with code reviews?” in 2015 IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering (SANER). IEEE, 2015, pp. 161–170. 10.1109/SANER.2015.7081826
- [89] F. Petersen, D. Mukherjee, Y. Sun, and M. Yurochkin, “Post-processing for individual fairness,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 34, pp. 25 944–25 955, 2021. <https://dl.acm.org/doi/10.5555/3540261.3542247>
- [90] M. Hassany, J. Ke, P. Brusilovsky, A. B. Lekshmi Narayanan, and K. Akhuseyinoglu, “Authoring worked examples for java programming with human ai collaboration,” in *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing*, 2024, pp. 101–103. 10.1145/3605098.3636160
- [91] H. Mozannar, G. Bansal, A. Fournay, and E. Horvitz, “Reading between the lines: Modeling user behavior and costs in ai-assisted programming,” in *Proceedings of the 2024 CHI conference on human factors in computing systems*, 2024, pp. 1–16. 10.1145/3613904.3641936
- [92] A. Stefik and S. Siebert, “An empirical investigation into programming language syntax,” *ACM Transactions on Computing Education*, vol. 13, no. 4, pp. 1–40, 2013. (TOCE) 10.1145/2534973